

جزوه امتحانی – داده‌های حجیم (Big Data Analytics)

استاد: دکتر فرساد زمانی بروجنی | امتحان: جزوه‌باز – ۹۰ دقیقه | منبع: لکچرهای ترم + نمونه سوالات + MMDS

روش پاسخ: تعریف یک خطی ← فرمول داخل باکس ← مثال عددی کوتاه. اول **فهرست صفحات** و **چیت‌شیت** را ببین. اگر سوال شبیه نمونه استاد بود → بخش 10.

فهرست صفحات (شماره واقعی همین PDF) [INDEX]

در امتحان برو به **شماره صفحه** ستون آخر. این شماره‌ها با پاصفحه PDF یکی هستند.

کلمه کلیدی سوال	بخش	چه بنویسی	صفحه
چیت‌شیت و فرمول‌های حیاتی	چیت‌شیت	تعریف + فرمول	1
TF-IDF / Collaborative Filtering	بخش 1	فرمول + مثال عددی	2
Cluster ↔ RDBMS / Hadoop	بخش 2	جدول تفاوت‌ها	3
MapReduce / Combiner / Pregel	بخش 3	۳ مرحله + خرابی	4
Join / جبر رابطه‌ای	بخش 4	مثال Join	5
Matrix × Vector	بخش 5	Map/Reduce + عدد	5
Shingle / Jaccard / MinHash / LSH / PCY	بخش 6	تعریف + فرمول + مثال	6
فاصله‌ها / اسمی / دودویی	بخش 7	فرمول + مثال	8
Apriori / Closed / Maximal	بخش 8	Pass ها + $M \subseteq C \subseteq F$	9
Clustering / K-Means / DBSCAN / BFR	بخش 9	مثال یک تکرار	9
حل نمونه سوالات استاد	بخش 10	پاسخ آماده	10

کل صفحات PDF: 11 | چیت‌شیت → ص 1 | بخش 1→2 ص 2 · بخش 2→3 ص 3 · بخش 3→4 ص 4 · بخش 4→5 ص 5 · بخش 5→6 ص 6 · بخش 6→7 ص 7 · بخش 7→8 ص 8 · بخش 8→9 ص 9 · بخش 9→10 ص 10 · بخش 10→11 ص 11

چیت‌شیت سریع [CHEAT]

مفهوم	تعریف یک خطی (همین را بنویس)
Shingle	دنباله k حرف/کلمه پشت‌سرهم در سند
Jaccard	اندازه اشتراک دو مجموعه ÷ اندازه اجتماع
MinHash	احتمال برابر شدن hash دو مجموعه = Jaccard
LSH	فقط جفت‌های احتمالاً مشابه را candidate می‌کند
Collaborative Filtering	توصیه بر اساس کاربران یا آیتم‌های شبیه
Closed itemset	پرتکرار؛ هیچ ابرمجموعه‌ای با همان support ندارد
Maximal itemset	پرتکرار؛ هیچ ابرمجموعه پرتکراری ندارد
MapReduce	پردازش موازی با Map و Reduce + تحمل خرابی

$$TF_{ij} = f_{ij} / \max_k(f_{kj})$$

$$IDF_i = \log_2(N / n_i)$$

$$TF-IDF = TF_{ij} \times IDF_i$$

که در آن:

f_{ij} = تعداد تکرار کلمه i در سند j

$\max_k(f_{kj})$ = بیشترین تکرار هر کلمه در همان سند j

N = تعداد کل اسناد

n_i = تعداد اسنادی که کلمه i در آن‌ها آمده

$$J(A, B) = |A \cap B| / |A \cup B|$$

$$\Pr[h\pi(C_1) = h\pi(C_2)] = J(C_1, C_2)$$

$$P(\text{candidate}) = 1 - (1 - s^r)^b$$

$$L_2 = \sqrt{\sum (x_i - y_i)^2} \quad L_1 = \sum |x_i - y_i|$$

$$L_\infty = \max |x_i - y_i| \quad \text{اسمی: } d = (p - m) / p$$

$$\text{support}(A \Rightarrow B) = \text{support}(A \cup B)$$

$$\text{confidence}(A \Rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$$

$$M \subseteq C \subseteq F$$

$$\text{Map: } (i, m_{ij} \times v_j) \quad \text{Reduce: } u_i = \sum_j (m_{ij} \times v_j)$$

مقایسه	نکته طلایی
RDBMS ↔ Cluster	عمودی/ACID ↔ افقی/commodity و انتقال کد به داده
K-Means ↔ K-Medoids	میانگین (حساس به پرت) ↔ نقطه واقعی داده
Combiner	فقط اگر Reduce انجمنی و جابجایی‌پذیر باشد

بخش 1 — مقدمه Big Data و TF-IDF [S1]

سه V

• **Volume:** حجم خیلی زیاد

• **Velocity:** تولید با سرعت بالا

• **Variety:** شکل‌های مختلف داده

داده‌کاوی: استخراج الگو/دانش مفید از داده.

دو تکنیک: خلاصه‌سازی (PageRank، خوشه‌بندی) و استخراج ویژگی (مشابهت، frequent itemsets، توصیه‌گر).

PageRank: صفحه مهم است اگر از صفحات مهم لینک بگیرد.

اصل Bonferroni: در داده عظیم، رخداد خیلی نادر هم زیاد دیده می‌شود → خطر False Positive.

Hash: $h(x) = x \bmod B$ | **Index:** بازیابی سریع رکورد | **دیسک:** خیلی کندتر از RAM؛ الگوریتم باید I/O کم کند.

کلمه مفید = در تعداد کمی سند، زیاد تکرار شده. کلماتی مثل «و» یا the که همه جا هستند، مفید نیستند (stop words).

$$TF_{ij} = f_{ij} / \max_k(f_{kj})$$

$$IDF_i = \log_2(N / n_i)$$

$$TF-IDF = TF_{ij} \times IDF_i$$

مثال 1 – دقیقاً مثل سوال استاد (قدم به قدم)

فرض: تعداد کل اسناد $N = 1024$

کلمه w در 128 سند آمده ($n = 128$)

در سند z این کلمه 10 بار آمده

پرتکرارترین کلمه همان سند 100 بار آمده

1 گام $TF = 10 / 100 = 0.1$

2 گام $IDF = \log_2(1024 / 128) = \log_2(8) = 3$

چون $8 = 3^2$

3 گام $TF-IDF = 0.1 \times 3 = 0.3$

پاسخ نهایی: 0.3

مثال 2 – برای فهم بهتر

اگر کلمه‌ای در همه اسناد باشد: $IDF = \log_2(1) = 0$ بی‌اهمیت.

کلمه «هادوپ» فقط در 1 سند از 3 سند آمده، 8 بار؛ $\max$ همان سند = 20:

$TF = 8/20 = 0.4$

$IDF = \log_2(3/1) \approx 1.58$

$TF-IDF \approx 0.4 \times 1.58 = 0.63$

پاسخ نهایی: ≈ 0.63

Collaborative Filtering

- **User-based**: کاربران شبیه تو چه چیزهایی را پسندیده‌اند؟ همان‌ها را به تو پیشنهاد بده.
- **Item-based**: آیتم‌های شبیه آنچه پسندیده‌ای را پیشنهاد بده.

در درس به Recommendation Engine (مثل Netflix و Amazon) و Behavioral Data اشاره شده.

بخش 2 – Cluster Computing در برابر RDBMS و Hadoop [S2]

محدودیت‌های RDBMS: مقیاس‌پذیری گران، سرعت، تحمل خرابی، پردازش پیشرفته روی ترابایت/پتابایت.

ویژگی‌های Cluster (حفظ کن): High Availability, Fault Tolerance, Data Replication, Load Balancing, Automated Failover, Backup & Recovery, Monitoring, Scalability.

موضوع	RDBMS سنتی	Cluster / Hadoop
سخت‌افزار	سرور گران	commodity ارزان
مقیاس	عمودی (تقویت یک سرور)	افقی (افزودن نود)
تحمل خرابی	غالباً سخت‌افزاری	نرم‌افزاری با replication
ایده پردازش	داده را پیش کد ببر	کد را پیش داده ببر

نمونه پاسخ آماده برای امتحان

- (۱) RDBMS برای داده ساخت یافته و تراکنش ACID عالی است، ولی وقتی داده خیلی بزرگ شود هزینه و سرعت مشکل می شود.
- (۲) Cluster Computing با سرورهای ارزان، کپی داده و failover خودکار، مقیاس افقی می دهد.
- (۳) جمله طلایی: در Hadoop به جای کشیدن همه داده به یک ماشین، برنامه را به جایی می فرستیم که داده هست.

Hadoop: Scalable + Fault Tolerant | HDFS + MapReduce | NameNode / DataNode / JobTracker / TaskTracker | معمولاً هر chunk سه کپی دارد.

SQL در برابر NoSQL: SQL اسکیمای ثابت و ACID؛ NoSQL اسکیمای انعطاف پذیر و مقیاس افقی.

بخش 3 — MapReduce [S3]

تو فقط دو تابع می نویسی؛ سیستم بقیه کار را می کند.

1. **Map:** هر تکه ورودی را بخوان و جفت های (کلید، مقدار) بساز.
2. **Shuffle:** همه مقدارهایی که کلید یکسان دارند را در یک لیست بگذار.
3. **Reduce:** برای هر کلید، لیست مقدارها را ترکیب کن (جمع، شمارش، ...).

Map: input $\rightarrow$ (key, value)

Shuffle: (k,v1), (k,v2), ... $\rightarrow$ (k, [v1, v2, ...])

Reduce: (k, [v1, v2, ...]) $\rightarrow$ (k, result)

مثال خیلی ساده — شمارش کلمه

دو جمله داریم: «big data» و «big data systems»

Map 1 جمله 1: (big,1) (data,1)
Map 2 جمله 2: (big,1) (data,1) (systems,1)

بعد از Shuffle:
big $\rightarrow$ [1, 1]
data $\rightarrow$ [1, 1]
systems $\rightarrow$ [1]

Reduce (جمع):
big = 2
data = 2
systems = 1

پاسخ نهایی: big=2 ، data=2 ، systems=1

Combiner: اگر عمل Reduce جابجایی پذیر و انجمنی باشد (مثل جمع)، می توان قبل از شبکه یک Reduce محلی زد تا ترافیک کم شود.

اگر این خراب شود	چه کار می کنند؟
Master	معمولاً کل Job از نو شروع می شود
Map Worker	همان Map ها روی نود سالم دوباره اجرا می شوند
Reduce Worker	Reduce دوباره اجرا می شود

Workflow Systems: به جای فقط Map $\rightarrow$ Reduce، یک گراف DAG از توابع (Clustera, Hyracks).

Pregel: سیستم محاسبات گرافی با SuperStep و Checkpoint برای تحمل خرابی در الگوریتم های بازگشتی.

PageRank تکراری:

تا وقتی تغییر خیلی کم شود $v^{(t+1)} = M \times v^{(t)}$

بخش 4 – جبر رابطه‌ای با MapReduce [S4]

عملیات	ایده ساده
Selection	فقط تاپل‌هایی که شرط دارند را نگه دار
Projection	بعضی ستون‌ها را بردار؛ تکراری‌ها را در Reduce حذف کن
Union	همه را بفرست؛ تکراری یکی شود
Intersection	فقط چیزی که در هر دو آمده
Difference R-S	چیزی که فقط در R است
Join	کلید = ستون مشترک؛ دو طرف را به هم بچسبان
Group + Aggregation	کلید = گروه؛ Reduce = SUM/COUNT/AVG

مثال Join برای مبتدی

جدول R: (علی، تهران) و (مینا، اصفهان)
 جدول S: (تهران، ایران) و (اصفهان، ایران)
 می‌خواهیم روی «شهر» Join کنیم:

Map: کلید = شهر

ایران S: علی و از R: تهران → از
 ایران S: مینا و از R: اصفهان → از

تهران: (علی، تهران، ایران) Reduce
 اصفهان: (مینا، اصفهان، ایران) Reduce

پاسخ نهایی: (علی، تهران، ایران) و (مینا، اصفهان، ایران)

Multi-way Join: Join همزمان چند جدول. هزینه انتقال داده مهم است؛ Replication Rate نشان می‌دهد هر ورودی چند بار کپی می‌شود.

بخش 5 – ضرب ماتریس با MapReduce [S5]

هدف ماتریس $\times$ بردار: برای هر سطر ماتریس یک عدد بساز = ضرب داخلی آن سطر در بردار.

$$u_i = m_{i1} \cdot v_1 + m_{i2} \cdot v_2 + \dots + m_{in} \cdot v_n$$

Map: $(i, m_{ij} \times v_j)$

Reduce: همه مقادیر را جمع کن i برای هر

مثال کامل – انگار اولین بار می‌بینی

ماتریس و بردار:

$$M = \begin{bmatrix} 1 & 0 & 5 \\ 7 & 3 & 0 \\ 1 & 3 & 5 \end{bmatrix} \quad V = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

گام Map: هر خانه ماتریس را در عنصر متناظر بردار ضرب کن. کلید = شماره سطر.

سطر 1: $(2=2 \times 1, 1)$, $(0=4 \times 0, 1)$, $(30=6 \times 5, 1)$
سطر 2: $(14=2 \times 7, 2)$, $(12=4 \times 3, 2)$, $(0=6 \times 0, 2)$
سطر 3: $(2=2 \times 1, 3)$, $(12=4 \times 3, 3)$, $(30=6 \times 5, 3)$

گام Shuffle: مقدارهای هر سطر را کنار هم بگذار.

کلید 1 $\rightarrow [30, 0, 2]$
کلید 2 $\rightarrow [0, 12, 14]$
کلید 3 $\rightarrow [30, 12, 2]$

گام Reduce: جمع کن.

$$\begin{aligned} u_1 &= 2+0+30 = 32 \\ u_2 &= 14+12+0 = 26 \\ u_3 &= 2+12+30 = 44 \end{aligned}$$

پاسخ نهایی: $U = [32, 26, 44]$

ماتریس $\times$ ماتریس (دو Job)

$$p_{ik} = \sum_j (m_{ij} \times n_{jk})$$

Job1: روی j جوین کن و ضرب‌های جزئی بساز. Job2: برای هر (i,k) جمع بزن.

چک یک خانه

$$\text{از } P: 1 \times 2 + 0 \times 4 + 5 \times 6 = 2+0+30 = 32 \text{ خانه } (1,1)$$

بخش 6 — Shingle , MinHash , LSH [S6]

Document $\rightarrow$ Shingles $\rightarrow$ MinHash Signature $\rightarrow$ LSH $\rightarrow$ Candidates $\rightarrow$ Exact check

Shingle چیست؟

یک پنجره به طول k روی متن می‌گذاری و هر بار یک تکه برمی‌داری.

مثال

متن: abcab و $k = 2$

تکه‌ها: ab , bc , ca , ab
مجموعه بدون تکرار: {ab, bc, ca}

$$J(A,B) = |A \cap B| / |A \cup B|$$

مثال با میوه

$A = \{\text{سیب، پرتقال، موز}\}$ و $B = \{\text{موز، انگور}\}$

1 → اشتراک = {موز}
 4 → اجتماع = {سیب، پرتقال، موز، انگور}
 $J = 1/4 = 0.25$
 فاصله = $1 - 0.25 = 0.75$

پاسخ نهایی: $J = 0.25$

MinHash

عناصر را تصادفی مرتب کن. برای هر مجموعه، اولین عنصری که در آن ترتیب می‌بینی را به عنوان hash بردار.

همان دو مجموعه Jaccard = [هاش دو مجموعه برابر شود] Pr

مثال

$A = \{1,3,4\}$ و $B = \{2,3,4,5\}$

واقعی $Jaccard\ 0.4 = 2/5$

یک ترتیب تصادفی فرضی: 3, 1, 5, 2, 4

در این ترتیب A 3 اولین عنصر
 در این ترتیب B 3 اولین عنصر
 ها برابر شدند hash پس

اگر خیلی ترتیب تصادفی بگیری، نسبت دفعاتی که برابر می‌شوند حدود 0.4 می‌شود.

LSH

امضا را به چند نوار (band) تقسیم کن. اگر دو سند حتی در یک نوار کاملاً یکسان بودند، آن‌ها را candidate حساب کن و بعد دقیق چک کن.

$$P = 1 - (1 - s)^b$$

که در آن:

s = شباهت واقعی

r = تعداد سطر در هر band

b = تعداد bandها

مثال عددی

$b = 20, r = 5, s = 0.8$

$s^r = 0.8^5 \approx 0.328$
 $(1 - 0.328)^{20} = 0.672^{20} \approx 0.00035$
 $P = 1 - 0.00035 \approx 0.99965$

یعنی حدود 99.97٪ احتمال دارد جفت واقعاً مشابه را پیدا کنیم.

پاسخ نهایی: $P \approx 0.99965$

PCY: در گذر اول علاوه بر شمارش اقلام، جفت‌ها را به bucket هش کن. در گذر دوم فقط جفتی را بشمار که هر دو قلم frequent باشند و bucketشان هم frequent باشد.

بخش 7 – فاصله‌ها و تشابه تاپل‌ها [S7]

چهار شرط Metric: نامنفی، صفر فقط وقتی دو شیء یکی باشند، تقارن، نامساوی مثلثی.

$$L_2 \text{ (اقلیدسی)} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots}$$

$$L_1 \text{ (منهتن)} = |x_1 - y_1| + |x_2 - y_2| + \dots$$

$$L_\infty = \max_i |x_i - y_i| \text{ بزرگ‌ترین}$$

مثال روی یک جفت نقطه

$X = (1, 2)$ و $Y = (4, 6)$

$$L_2 = \sqrt{(1-4)^2 + (2-6)^2} = \sqrt{9+16} = \sqrt{25} = 5$$

$$L_1 = |1-4| + |2-6| = 3 + 4 = 7$$

$$L_\infty = \max(3, 4) = 4$$

$$d = (p - m) / p \text{ : برای صفات اسمی}$$

که در آن:

p = تعداد کل صفات

m = تعداد صفاتی که مقدارشان یکی است

مثال اسمی

شخص 1: (مرد، تهران، لیسانس)

شخص 2: (زن، تهران، لیسانس)

$$p = 3$$

$$m = 2 \text{ (شهر و مدرک یکسان‌اند)}$$

$$d = (3-2)/3 = 1/3 \approx 0.33$$

پاسخ نهایی: 1/3

دودویی: $q = \text{هر دو} \mid t \mid s = (1,0) \mid r = (0,1) = \text{هر دو} = 0$

$$d = (r + s) / (q + r + s + t) \text{ : متقارن}$$

$$d = (r + s) / (q + r + s) \leftarrow \text{صفر-صفر مهم نیست : نامتقارن}$$

$$\text{Jaccard: } J = q / (q + r + s)$$

Cosine برای متن خوب است. **Hamming** = تعداد خانه‌های متفاوت. **Edit distance** = $|X| + |Y| - 2 \times |\text{LCS}|$.

Data Matrix: سطر = تاپل، ستون = صفت. **Dissimilarity Matrix**: فاصله بین تاپل‌ها؛ قطر اصلی صفر.

$\text{support}(X) = (\text{تعداد تراکنش شامل } X) / (\text{کل تراکنش‌ها})$

$\text{support}(A \Rightarrow B) = \text{support}(A \cup B)$

$\text{confidence}(A \Rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$

$M \subseteq C \subseteq F$

مثال قانون انجمنی

از 100 سبد خرید: نان در 40 سبد، نان+شیر در 30 سبد.

قانون: نان $\Rightarrow$ شیر

$\text{support} = 30/100 = 0.3$

$\text{confidence} = 30/40 = 0.75 = 75\%$

یعنی 75٪ کسانی که نان گرفته‌اند شیر هم گرفته‌اند.

پاسخ نهایی: $\text{conf} = 75\%$

نوع	معنی ساده
Closed	اگر قلمی اضافه کنی، support دیگر همان عدد قبلی نمی‌ماند
Maximal	اگر قلمی اضافه کنی، دیگر frequent نیست

مثال Closed و Maximal

تراکنش‌ها: $\{A,B\}$ ، $\{A,B,C\}$ ، $\{A,C\}$ و حداقل شمارش $= 2$

$A=4$ ، $B=3$ ، $C=3$

$AB=3$ ، $AC=3$ ، $BC=2$

$ABC=2$

Closed ها: A ، AB ، AC ، ABC

(را دارد $\text{support}=3$ همان AB نیست چون B closed)

Maximal فقط: ABC

اصل Apriori: اگر مجموعه‌ای frequent باشد، همه زیرمجموعه‌هایش هم frequent اند. پس کاندید بزرگ را فقط از frequent های کوچک می‌سازیم.

CHARM: الگوریتم استخراج Closed با اشتراک tidsetها.

بخش 9 — خوشه‌بندی [S9]

هدف: اعضای یک خوشه به هم نزدیک؛ خوشه‌های مختلف از هم دور. یادگیری بدون سرپرست (برخلاف طبقه‌بندی).

انواع: Partitioning (K-Means, K-Medoids, PAM, CLARA, CLARANS) · Hierarchical (AGNES, DIANA) · Density (DBSCAN, OPTICS, DENCLUE) · Grid (STING, CLIQUE)

مثال K-Means – فقط یک تکرار

نقاط: $A(1,1)$, $B(1,2)$, $C(4,4)$, $D(5,4)$

مراکز اولیه: $\mu_1 = A(1,1)$ و $\mu_2 = C(4,4)$

تخصیص: هر نقطه به نزدیک‌ترین مرکز می‌رود.

$C_1 = \{A, B\}$ → هستند μ_1 نزدیک B و A

$C_2 = \{C, D\}$ → هستند μ_2 نزدیک D و C

به‌روزرسانی مرکز: میانگین بگیر.

$\mu_1 = ((1+1)/2, (1+2)/2) = (1, 1.5)$

$\mu_2 = ((4+5)/2, (4+4)/2) = (4.5, 4)$

پاسخ نهایی: مراکز جدید: $(1.5, 1)$ و $(4, 4.5)$

Linkage: Single = کمینه فاصله بین دو خوشه · Complete = بیشینه · Average = میانگین · Centroid = فاصله مراکز.
K-Medoids / PAM: نماینده خوشه یک نقطه واقعی است (مقاوم‌تر به پرت). CLARA روی نمونه کار می‌کند؛ CLARANS جستجوی تصادفی می‌کند.

Core Point → نقطه باشد **MinPts** حداقل **Eps** اگر در شعاع **DBSCAN**:

نیست **Core** است ولی خودش **Core** کنار **Border** =

هیچ‌کدام **Noise** =

BFR (داده بزرگ اقلیدسی):

$$\text{centroid}_i = \text{SUM}_i / N$$

$$\text{variance}_i = \text{SUMSQ}_i / N - (\text{SUM}_i / N)^2$$

CURE: چند نقطه نماینده برای خوشه‌های غیرکروی. **OPTICS**: وقتی چگالی خوشه‌ها فرق می‌کند، بهتر از DBSCAN است.

بخش 10 – حل کامل نمونه سوالات استاد [S10]

امتحان نمونه: ۹۰ دقیقه — ۱۰ نمره توصیفی

سوال 1 – تعاریف

الف) Min-hashing: از مجموعه بزرگ امضای کوتاه می‌سازیم طوری که احتمال برابر شدن hash دو مجموعه برابر Jaccard باشد.

$$\Pr[h(C_1) = h(C_2)] = J(C_1, C_2)$$

ب) Collaborative Filtering: توصیه‌گر مبتنی بر شباهت کاربران یا آیتم‌ها (مثل Netflix/Amazon).
ج) Jaccard:

$$J(A, B) = |A \cap B| / |A \cup B|$$

د) Shingle: دنباله k توکن متوالی در سند.

سوال 2 – Cluster در برابر RDBMS

RDBMS: مقیاس غالباً عمودی، ACID، داده ساخت یافته، در حجم خیلی بالا گران/کند.
Cluster: مقیاس افقی، replication، commodity servers، failover، انتقال کد به سمت داده.

سوال 3 – MapReduce ب Matrix×Vector

Map: $(i, m_{ij} \times v_j)$

Reduce: $u_i =$ مجموع همه مقادیرهای کلید i

مثال عددی کامل در بخش 5.

سوال 4 – TF-IDF

حل سوال استاد

$$TF = 10/100 = 0.1$$

$$IDF = \log_2(1024/128) = \log_2(8) = 3$$

$$TF-IDF = 0.1 \times 3 = 0.3$$

پاسخ نهایی: 0.3

سوال 5 – Apriori ب MST = 0.3

TID	افلام	TID	افلام
1	a, b	5	a, b, c
2	b, c	6	d, f
3	a, b, c	7	c, d, e, f
4	d, e, f	8	a, b, c, d, e

حل قدم به قدم

تعداد تراکنش = 8 و $\text{minsup} = 0.3 \rightarrow$ حداقل شمارش باید $3 \leq$ باشد (چون $2/8 = 0.25$ از 0.3 کمتر است).

Pass 1:

$$a=4, b=5, c=5, d=4, e=3, f=3$$

$$L1 = \{a, b, c, d, e, f\} \rightarrow \text{همه } 3 \leq$$

Pass 2:

$$ab=4 \checkmark \quad ac=3 \checkmark \quad bc=4 \checkmark$$

$$de=3 \checkmark \quad df=3 \checkmark$$

$$cd=2 \times \quad ce=2 \times \quad ef=2 \times$$

$$L2 = \{ab, ac, bc, de, df\}$$

Pass 3: فقط abc قابل ساخت است (ab و ac و bc همه frequent اند). چون ef frequent نیست.

$$\text{count}=3 \checkmark \rightarrow \text{در تراکنشهای 3 و 5 و 8 abc}$$

$$L3 = \{abc\}$$

Pass 4 کاندیدی ندارد $\rightarrow$ توقف.

پاسخ نهایی: $L2 = \{ab, ac, bc, de, df\}$ و $L3 = \{abc\}$

چک لیست ۳۰ ثانیه‌ای: تعریف یک خطی؟ فرمول؟ مثال عددی؟ تفاوت با مفهوم نزدیک؟

موارد اضافه شده پس از مرور لکچرها: Hash/Index/دیسک، Data Streams، Bonferroni، Pregel، Multi-way Join، PCY، OPTICS/CURE/BFR. Online Advertising فقط در فهرست کلی overview بودند و اسلاید کامل در فایل‌های ارسالی نبود؛ تمرکز نمونه سوالات استاد روی Similar Items، MapReduce، TF-IDF، Cluster/RDBMS و Apriori است.